ECE3411

Lecture 14

SPI: Serial Peripheral Interface

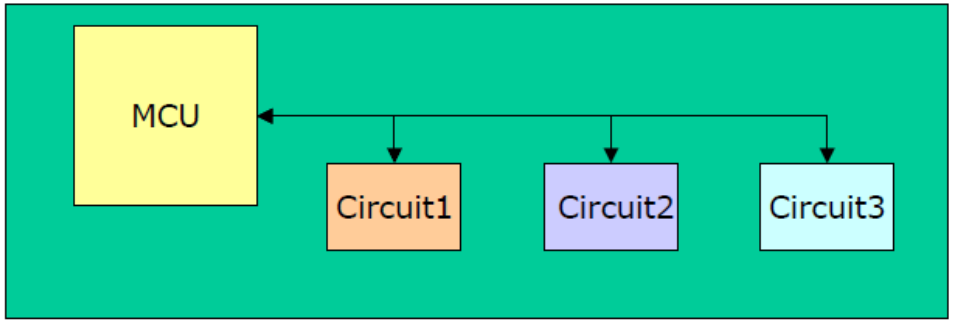
Sung Yeul Park
Department of Electrical & Computer Engineering
University of Connecticut
Email: sung_yeul.park@uconn.edu

Adapted from ECE3411 – Fall 2015, by Marten van Dijk and Syed Kamran Haider

Adapted from John Chandy ECE3411 – Fall 2024

Serial Peripheral Interfaces

- Local serial interconnection of microcontrollers and peripheral circuits/functions
- Required features:
  - Low complexity
  - Low to medium data rate
  - Small physical footprint/few pins
  - Short distances
  - Low cost
- Most MCUs have built-in peripheral units for communicating with external circuits, e.g. ATmegaAVR (SPI and TWI (I2C))
- Great abundance of different types of peripheral circuits that implements synchronous serial interfaces (Flash, EEPROM, AD, DA, RTC, Display drivers, sensors etc.)



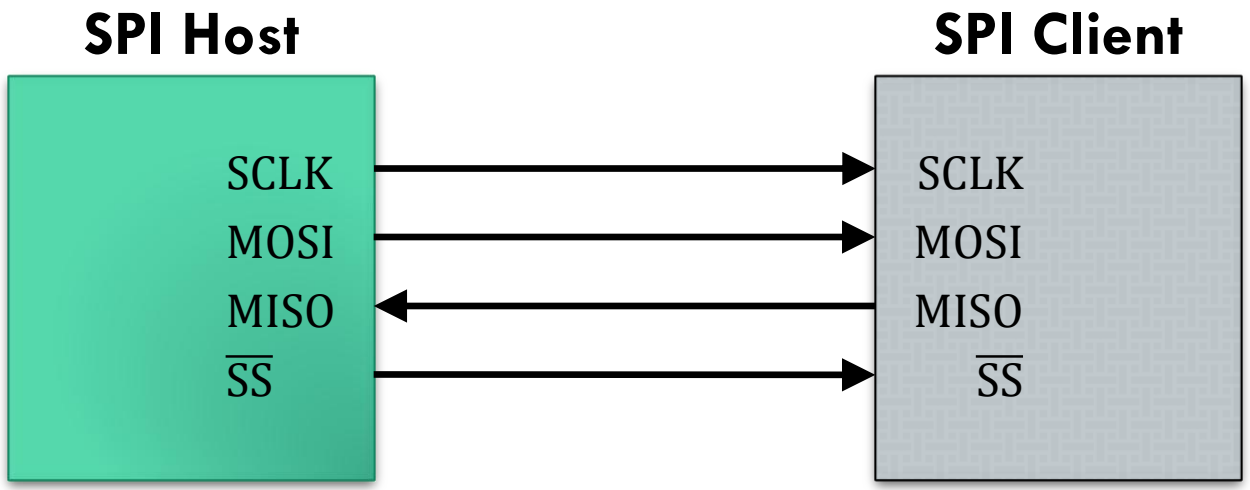
Top Results for: sensor SPI

Embedded - Microcontrollers (1741 items)	Integrated Circuits (ICs)
Motion Sensors - Accelerometers (571 items)	Sensors, Transducers
Pressure Sensors, Transducers (551 items)	Sensors, Transducers
Data Acquisition - Analog to Digital Converters (ADC) (433 items)	Integrated Circuits (ICs)
Evaluation Boards - Sensors (431 items)	Development Boards, Kits, Programmers
Temperature Sensors - Analog and Digital Output (355 items)	Sensors, Transducers
Motion Sensors - IMUs (Inertial Measurement Units) (253 items)	Sensors, Transducers
Interface - Sensor, Capacitive Touch (212 items)	Integrated Circuits (ICs)
Motion Sensors - Gyroscopes (109 items)	Sensors, Transducers
Position Sensors - Angle, Linear Position Measuring (107 items)	Sensors, Transducers
Magnetic Sensors - Linear, Compass (ICs) (101 items)	Sensors, Transducers
Specialized Sensors (59 items)	Sensors, Transducers
PMIC - Thermal Management (45 items)	Integrated Circuits (ICs)
Optical Sensors - Ambient Light, IR, UV Sensors (43 items)	Sensors, Transducers
Humidity, Moisture Sensors (41 items)	Sensors, Transducers

SPI: Serial Peripheral Interface

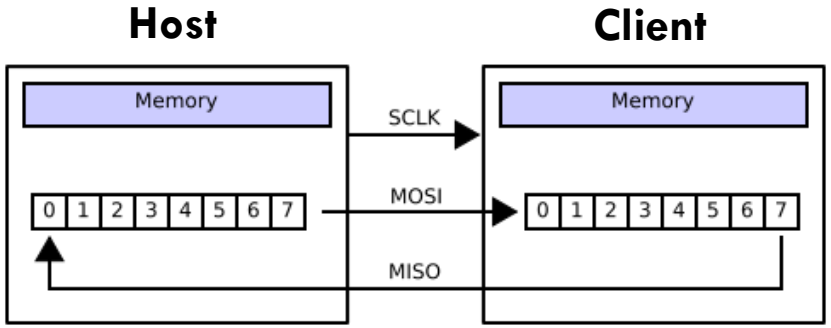
The SPI bus specifies four logic signals:

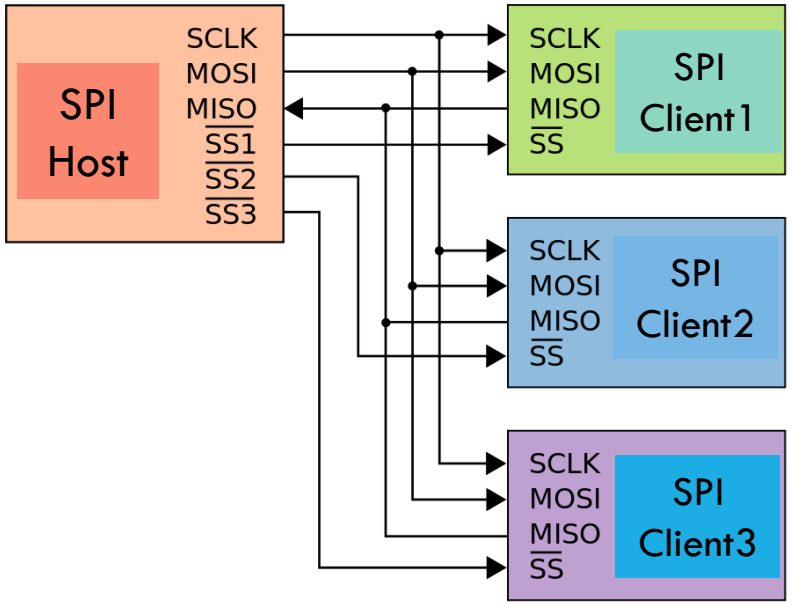
- SCLK : Serial Clock (output from host).
- MOSI : Host Output, Client Input (output from host).
- MISO : Host Input, Client Output (output from client).
- SS : Client Select (active low, output from host).



SPI Host & Clients

- The SPI bus can operate with a single host and with one or more clients.
- In case of multiple clients, the host selects the client with a logic 0 on the select (SS) line.
- During each SPI clock cycle, the host sends a bit on the MOSI line and the client reads it, while the client sends a bit on the MISO line and the host reads it.
- This sequence is maintained even when only one-directional data transfer is intended.





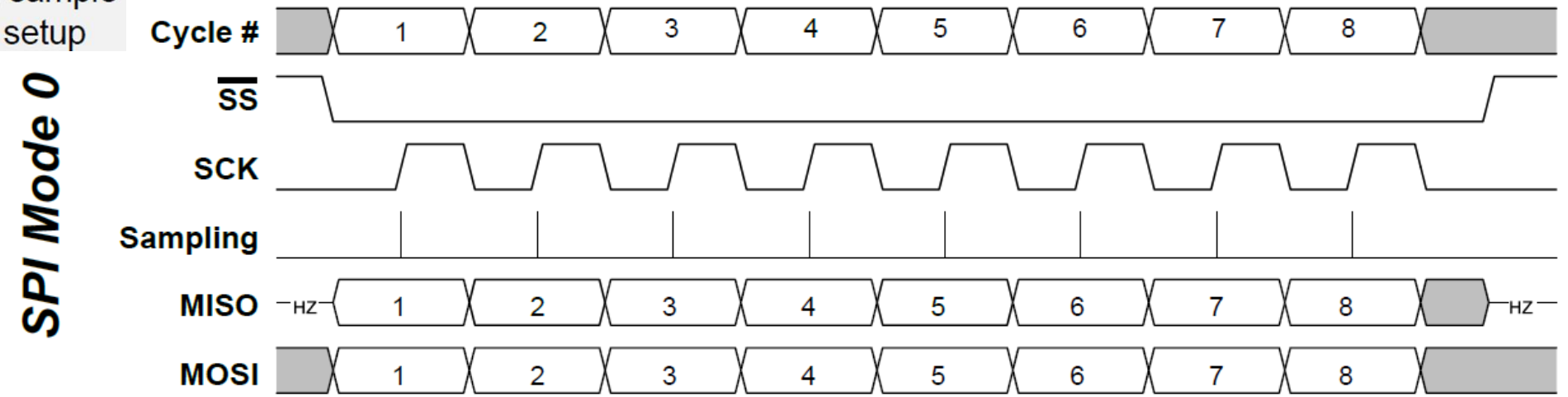
SPI Data Modes

- SPI Data Transfer Modes: There are four combinations of SCK phase and polarity with respect to serial data, which are determined by mode selection bits.

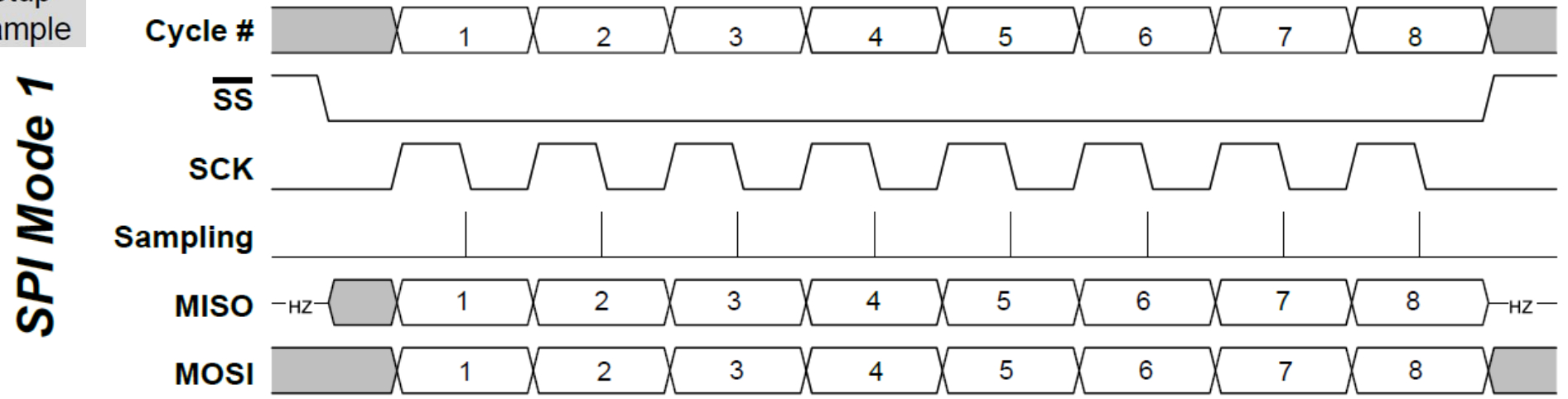
Value	Name	Description
0x0	0	Leading edge: Rising, sample Trailing edge: Falling, setup
0x1	1	Leading edge: Rising, setup Trailing edge: Falling, sample
0x2	2	Leading edge: Falling, sample Trailing edge: Rising, setup
0x3	3	Leading edge: Falling, setup Trailing edge: Rising, sample

SPI Data Transfer Modes

Leading edge: Rising, sample
Trailing edge: Falling, setup



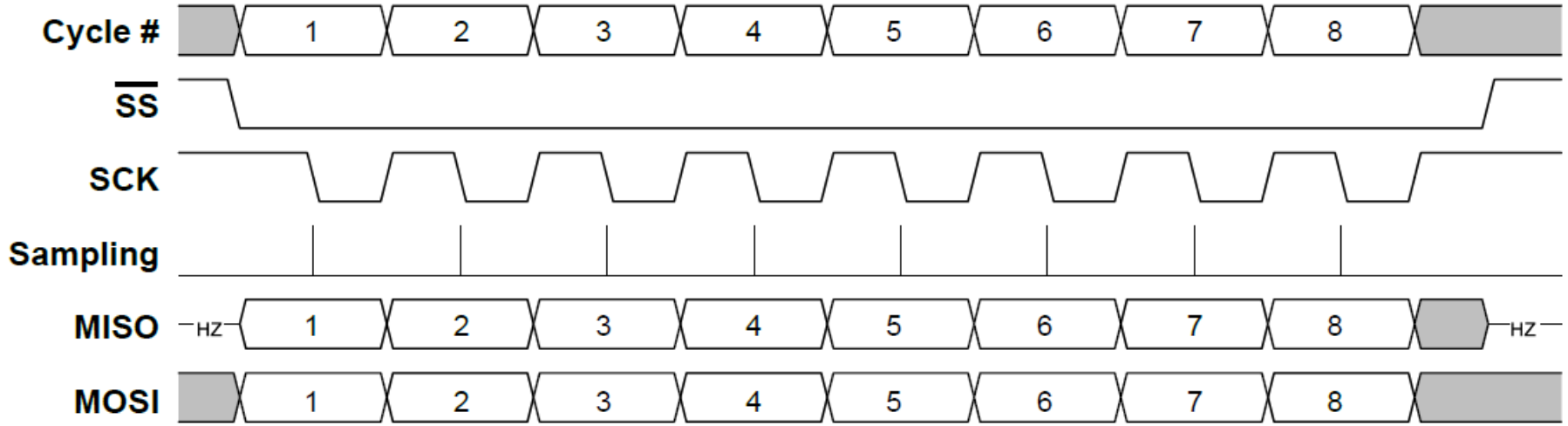
Leading edge: Rising, setup
Trailing edge: Falling, sample



SPI Data Transfer Modes

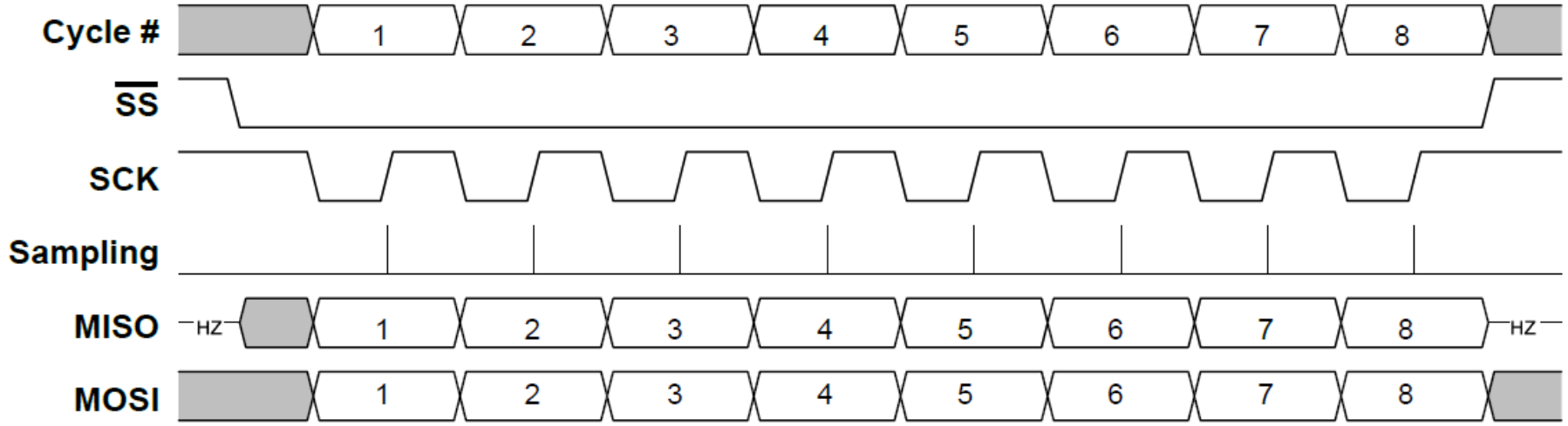
Leading edge: Falling, sample
Trailing edge: Rising, setup

SPI Mode 2



Leading edge: Falling, setup
Trailing edge: Rising, sample

SPI Mode 3



CTRLA Register

CTRLA
0x00
0x00

Bit	7	6	5	4	3	2	1	0
		DORD	MASTER	CLK2X		PRESC[1:0]		ENABLE
Access		R/W	R/W	R/W		R/W	R/W	R/W
Reset		0	0	0		0	0	0

Bit 6 – DORD Data Order

Value	Description
0	The MSb of the data word is transmitted first
1	The LSb of the data word is transmitted first

Bit 5 – MASTER Host/Client Select

This bit selects the desired SPI mode.
If $\overline{SS}$ is configured as input and driven low while this bit is '1', then this bit is cleared, and the IF in SPIn.INTFLAGS is set. The user has to write MASTER = 1 again to re-enable SPI Host mode.
This behavior is controlled by the Client Select Disable (SSD) bit in SPIn.CTRLB.

Value	Description
0	SPI Client mode selected
1	SPI Host mode selected

Bit 4 – CLK2X Clock Double

When this bit is written to '1', the SPI speed (SCK frequency, after internal prescaler) is doubled in Host mode.

Value	Description
0	SPI speed (SCK frequency) is not doubled
1	SPI speed (SCK frequency) is doubled in Host mode

CTRLA Register

CTRLA
0x00
0x00

Bit	7	6	5	4	3	2	1	0
		DORD	MASTER	CLK2X		PRESC[1:0]		ENABLE
Access		R/W	R/W	R/W		R/W	R/W	R/W
Reset		0	0	0		0	0	0

Bits 2:1 – PRESC[1:0] Prescaler

This bit field controls the SPI clock rate configured in Host mode. These bits have no effect in Client mode. The relationship between SCK and the peripheral clock frequency (f_{CLK_PER}) is shown below.
The output of the SPI prescaler can be doubled by writing the CLK2X bit to '1'.

Value	Name	Description
0x0	DIV4	CLK_PER/4
0x1	DIV16	CLK_PER/16
0x2	DIV64	CLK_PER/64
0x3	DIV128	CLK_PER/128

Bit 0 – ENABLE SPI Enable

Value	Description
0	SPI is disabled
1	SPI is enabled

CTRLB Register

Control B

Name: CTRLB
Offset: 0x01
Reset: 0x00

Bit	7	6	5	4	3	2	1	0
	BUFEN	BUFWR				SSD	MODE[1:0]	
Access	R/W	R/W				R/W	R/W	R/W
Reset	0	0				0	0	0

Bit 7 – BUFEN Buffer Mode Enable

Writing this bit to '1' enables Buffer mode. This will enable two receive buffers and one transmit buffer. Both will have separate interrupt flags, transmit complete and receive complete.

Bit 6 – BUFWR Buffer Mode Wait for Receive

When writing this bit to '0', the first data transferred will be a dummy sample.

Value	Description
0	One SPI transfer must be completed before the data are copied into the shift register
1	If writing to the Data register when the SPI is enabled and $\overline{SS}$ is high, the first write will go directly to the shift register

Bit 2 – SSD Client Select Disable

If this bit is set when operating as SPI Host (MASTER = 1 in SPIn.CTRLA), $\overline{SS}$ does not disable Host mode.

Value	Description
0	Enable the Client Select line when operating as SPI host
1	Disable the Client Select line when operating as SPI host

By default, the SPI hardware will control the slave select line. If you need to control it in software, set the SSD bit

Interrupt Control

Bit	7	6	5	4	3	2	1	0
	RXCIE	TXCIE	DREIE	SSIE				IE
Access	R/W	R/W	R/W	R/W				R/W
Reset	0	0	0	0				0

Bit 7 – RXCIE Receive Complete Interrupt Enable
In Buffer mode, this bit enables the Receive Complete interrupt. The enabled interrupt will be triggered when the RXCIF in the SPIn.INTFLAGS register is set. In the Non-Buffer mode, this bit is '0'.

Bit 6 – TXCIE Transfer Complete Interrupt Enable
In Buffer mode, this bit enables the Transfer Complete interrupt. The enabled interrupt will be triggered when the TXCIF in the SPIn.INTFLAGS register is set. In the Non-Buffer mode, this bit is '0'.

Bit 5 – DREIE Data Register Empty Interrupt Enable
In Buffer mode, this bit enables the Data Register Empty interrupt. The enabled interrupt will be triggered when the DREIF in the SPIn.INTFLAGS register is set. In the Non-Buffer mode, this bit is '0'.

Bit 4 – SSIE Slave Select Trigger Interrupt Enable
In Buffer mode, this bit enables the Slave Select interrupt. The enabled interrupt will be triggered when the SSIF in the SPIn.INTFLAGS register is set. In the Non-Buffer mode, this bit is '0'.

Bit 0 – IE Interrupt Enable
This bit enables the SPI interrupt when the SPI is not in Buffer mode. The enabled interrupt will be triggered when RXCIF/IF is set in the SPIn.INTFLAGS register.

Interrupt Flags – Normal Mode

Bit	7	6	5	4	3	2	1	0
	IF	WRCOL						
Access	R/W	R/W						
Reset	0	0						

Bit 7 – IF Interrupt Flag

This flag is set when a serial transfer is complete, and one byte is completely shifted in/out of the SPIn.DATA register. If $\overline{SS}$ is configured as input and is driven low when the SPI is in Host mode, this will also set this flag. The IF is cleared by writing a '1' to it. Alternatively, the IF can be cleared by first reading the SPIn.INTFLAGS register when IF is set and then accessing the SPIn.DATA register.

Bit 6 – WRCOL Write Collision

The WRCOL flag is set if the SPIn.DATA register is written before a complete byte has been shifted out. This flag is cleared by first reading the SPIn.INTFLAGS register when WRCOL is set and then accessing the SPIn.DATA register.

Interrupt Flags – Buffer Mode

Bit	7	6	5	4	3	2	1	0
	RXCIF	TXCIF	DREIF	SSIF				BUFOVF
Access	R/W	R/W	R/W	R/W				R/W
Reset	0	0	0	0				0

Bit 7 – RXCIF Receive Complete Interrupt Flag
This flag is set when there are unread data in the Receive Data Buffer register and cleared when the Receive Data Buffer register is empty (that is, it does not contain any unread data).
When interrupt-driven data reception is used, the Receive Complete Interrupt routine must read the received data from the DATA register in order to clear RXCIF. If not, a new interrupt will occur directly after the return from the current interrupt. This flag can also be cleared by writing a '1' to its bit location.

Bit 6 – TXCIF Transfer Complete Interrupt Flag
This flag is set when all the data in the Transmit shift register has been shifted out, and there is no new data in the transmit buffer (SPIn.DATA). The flag is cleared by writing a '1' to its bit location.

Bit 5 – DREIF Data Register Empty Interrupt Flag
This flag indicates whether the Transmit Data Buffer register is ready to receive new data. The flag is '1' when the transmit buffer is empty and '0' when the transmit buffer contains data to be transmitted that has not yet been moved into the shift register. The DREIF is cleared after a Reset to indicate that the transmitter is ready.
The DREIF is cleared by writing to DATA. When interrupt-driven data transmission is used, the Data Register Empty Interrupt routine must either write new data to DATA in order to clear DREIF or disable the Data Register Empty interrupt. If not, a new interrupt will occur directly after the return from the current interrupt.

Interrupt Flags – Buffer Mode

Bit	7	6	5	4	3	2	1	0
	RXCIF	TXCIF	DREIF	SSIF				BUFOVF
Access	R/W	R/W	R/W	R/W				R/W
Reset	0	0	0	0				0

Bit 4 – SSIF Slave Select Trigger Interrupt Flag
This flag indicates that the SPI has been in Master mode and the $\overline{SS}$ pin has been pulled low externally, so the SPI is now working in Slave mode. The flag will only be set if the Slave Select Disable (SSD) bit is not '1'. The flag is cleared by writing a '1' to its bit location.

Bit 0 – BUFOVF Buffer Overflow
This flag indicates data loss due to a Receive Data Buffer full condition. This flag is set if a Buffer Overflow condition is detected. A Buffer Overflow occurs when the receive buffer is full (two bytes), and a third byte has been received in the shift register. If there is no transmit data, the Buffer Overflow will not be set before the start of a new serial transfer. This flag is cleared when the DATA register is read, or by writing a '1' to its bit location.

Normal Mode vs Buffer Mode

Feature	Normal Mode	Buffer Mode
Data Handling	Byte-by-byte	Buffered (multi-byte capable)
CPU Load	Higher (frequent interrupts)	Lower (fewer interrupts)
Speed	Slower	Faster
Best For	Simple SPI tasks	High-speed or bulk transfers

SPI DATA Register

Bit	7	6	5	4	3	2	1	0
	DATA[7:0]							
Access	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset	0	0	0	0	0	0	0	0

Bits 7:0 – DATA[7:0] SPI Data

The DATA register is used for sending and receiving data. Writing to the register initiates the data transmission when in Host mode while preparing data for sending in Client mode. The byte written to the register shifts out on the SPI output line when a transaction is initiated.

The SPIn.DATA register is not a physical register. Depending on what mode is configured, it is mapped to other registers, as described below.

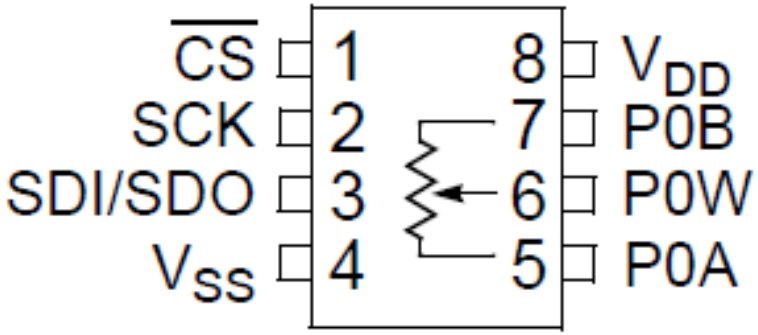
- Normal mode:
  - Writing the DATA register will write the shift register
  - Reading from DATA will read from the Receive Data register
- Buffer mode:
  - Writing the DATA register will write to the Transmit Data Buffer register
  - Reading from DATA will read from the Receive Data Buffer register. The contents of the Receive Data register will then be moved to the Receive Data Buffer register.

Digital Potentiometer

The MCP4131 allows you to configure the resistance of a potentiometer programmatically.

- 7-bit resolution.
- 50 kΩ maximum resistance

1	CS	Chip Select Input. (SPI Slave Select)
2	SCK	SPI Serial Clock Input
3	SDI	SPI Serial Data Input (MOSI)
4	V_{SS}	Ground
5	P0A	Terminal end of resistor network
6	P0W	Tap/Wiper of digital potentiometer
7	P0B	Terminal end of resistor network
8	V_{DD}	Positive Power Supply Input (1.8V to 5.5V)



Digital Potentiometer

Software can control which of the different taps is connected to the wiper (W) output

R_{AB} = 50$K\Omega$ (depends on which version of the MCP4131-xxx)

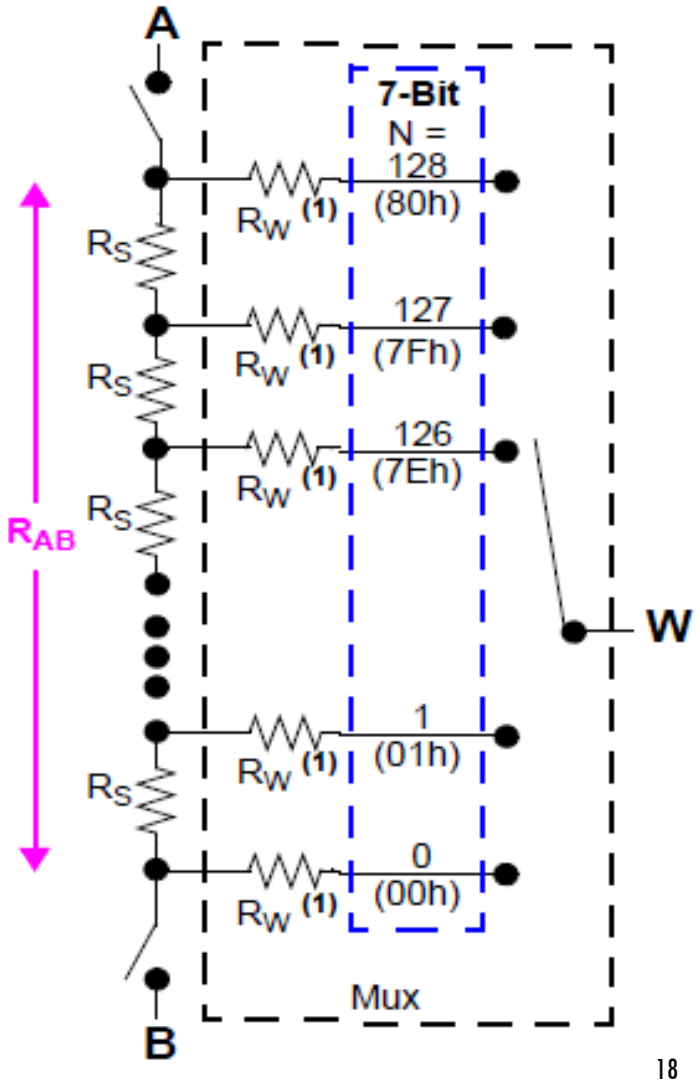
R_{WB} = 50$K\Omega$*$WIPER$/128 + R_W

R_{AW} = 50$K\Omega$*(128 −$WIPER$)/128 + R_W

R_W is around 75Ω

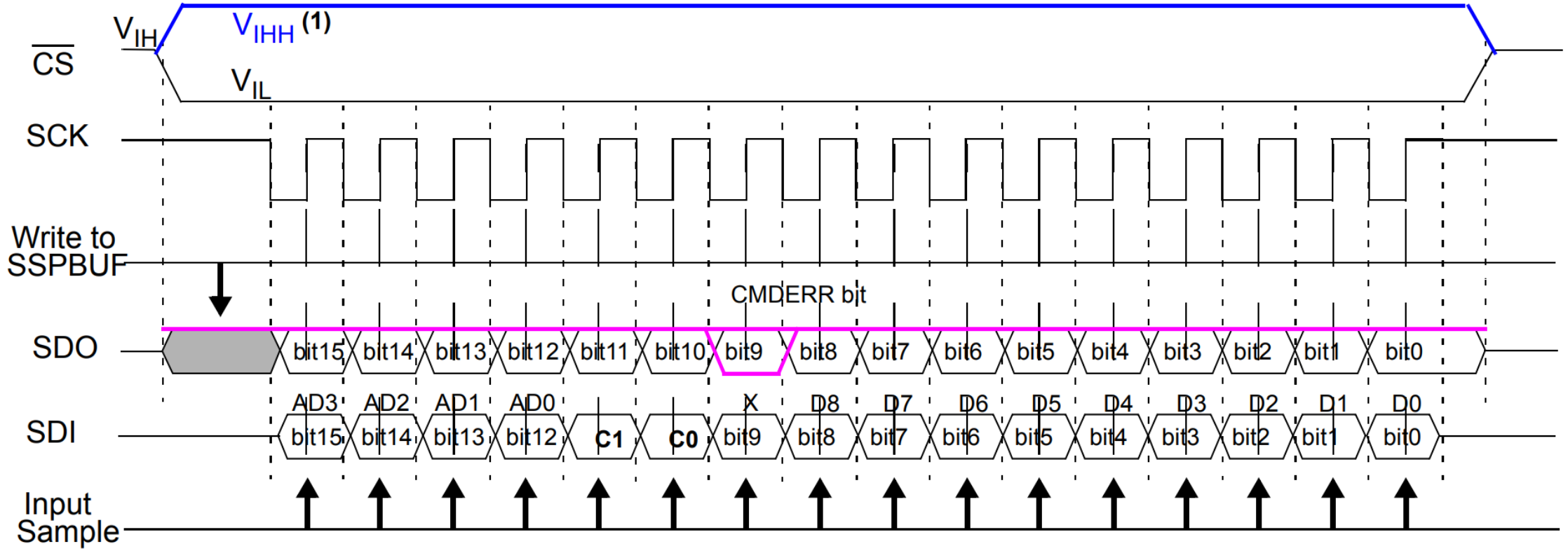
The A and B terminals can be disconnected from the resistor network if you just need one programmable resistor instead of a potentiometer configuration

Uses less current



SPI Frame Format and Timing

- MCP4131 receives a 16-bit word from the MCU in two combined 8-bit SPI transactions.
- The format of the 16-bit frame containing 4 address, 2 command, and 10 data bits is shown below
- MCP4131 use SPI Mode 0



MCP4131 Command Bits

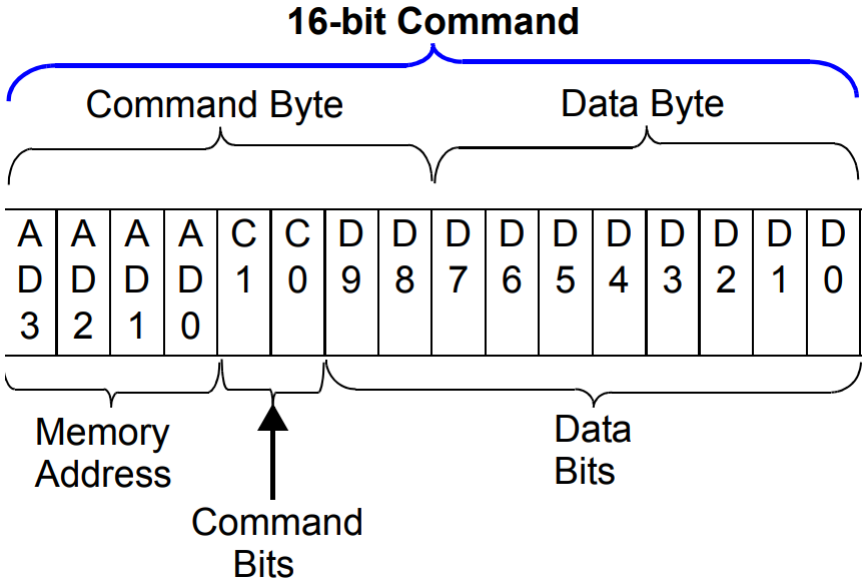


TABLE 4-1: MEMORY MAP

Address	Function	Memory Type
00h	Volatile Wiper 0	RAM
01h	Volatile Wiper 1	RAM
02h	Reserved	—
03h	Reserved	—
04h	Volatile TCON Register	RAM
05h	Status Register	RAM
06h-0Fh	Reserved	—

- MCP4131 receives a 16-bit word from the MCU in two combined 8-bit SPI transactions.
- The format of the 16-bit frame containing 4 address, 2 command, and 10 data bits is shown to the left.
- The upper 4 bits of the 16-bit word are the address of the register that you are writing to
  - The volatile wiper 0 register selects which tap you want to use
  - The TCON register enables/disables connections in the resistor network
- The command bits are “00” for write or “11” for read

MCP4131 Command Bits: Terminal Control Register

REGISTER 4-2: TCON BITS (1, 2)

R-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1	R/W-1
D8	R1HW	R1A	R1W	R1B	R0HW	R0A	R0W	R0B
bit 8								bit 0

bit 3 **R0HW:** Resistor 0 Hardware Configuration Control bit

This bit forces Resistor 0 into the "shutdown" configuration of the Hardware pin
1 = Resistor 0 is NOT forced to the hardware pin "shutdown" configuration
0 = Resistor 0 is forced to the hardware pin "shutdown" configuration

bit 2 **R0A:** Resistor 0 Terminal A (P0A pin) Connect Control bit

This bit connects/disconnects the Resistor 0 Terminal A to the Resistor 0 Network
1 = P0A pin is connected to the Resistor 0 Network
0 = P0A pin is disconnected from the Resistor 0 Network

bit 1 **R0W:** Resistor 0 Wiper (P0W pin) Connect Control bit

This bit connects/disconnects the Resistor 0 Wiper to the Resistor 0 Network
1 = P0W pin is connected to the Resistor 0 Network
0 = P0W pin is disconnected from the Resistor 0 Network

bit 0 **R0B:** Resistor 0 Terminal B (P0B pin) Connect Control bit

This bit connects/disconnects the Resistor 0 Terminal B to the Resistor 0 Network
1 = P0B pin is connected to the Resistor 0 Network
0 = P0B pin is disconnected from the Resistor 0 Network

Sample Code

```
#define F_CPU 16000000UL
#include <avr/io.h>
#include <util/delay.h>

// MCP4131 Commands
#define MCP4131_WRITE_CMD 0x00  // Write command to
potentiometer
#define MCP4131_WIPER0    0x00  // Target wiper 0

void SPI1_init(void) {
// Route SPI1 to Port C: PC4 (MOSI), PC6 (SCK)
PORTMUX.SPIROUTEA = PORTMUX_SPI1_ALT1_gc;

// Enable SPI1 in Master mode, Prescaler = 128
SPI1.CTRLA = SPI_ENABLE_bm | SPI_MASTER_bm |
SPI_PRESC_DIV128_gc;

// Enable SS pin control (optional)
SPI1.CTRLB = SPI_SSD_bm;
}
```

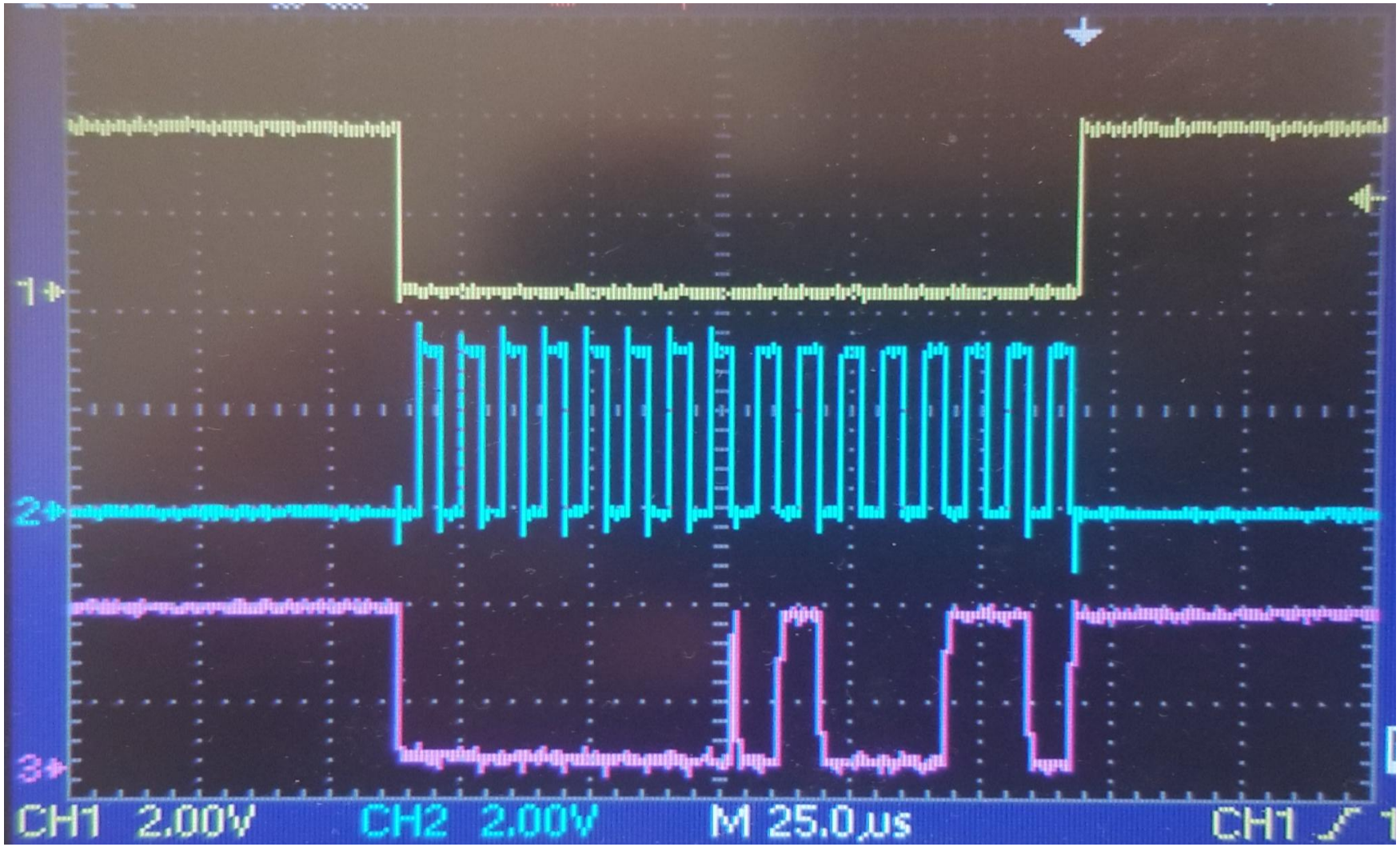
```
void SPI1_write(uint8_t data) {
SPI1.DATA = data;
while (!(SPI1.INTFLAGS & SPI_IF_bm)); // Wait for
transfer complete
}

void MCP4131_setWiper(uint8_t value) {
PORTC.OUTCLR = PIN7_bm; // CS low
SPI1_write(MCP4131_WRITE_CMD | MCP4131_WIPER0); // Send
command
SPI1_write(value); // Send wiper value (0-255)
PORTC.OUTSET = PIN7_bm; // CS high
}

int main(void) {
CLK_init();
SPI_init();

while (1) {
    for (uint8_t i = 0; i < 129; i += 1) {
    MCP4131_setWiper(i);
    }
  }
}
```

SPI Waveform



CS Signal

SCK Signal

MOSI Signal

Notes

- The MCP4131 requires 16 bits to be sent in one SPI transaction
- Modify the SPI_Master_Transceiver code to take a 16-bit input instead of an 8-bit input.
- Instead of sending one 8-bit value, send two 8-bit values
- First, pull down SS/CS in software
- Each 8-bit data is sent by putting it into DATA (which starts the SPI send process) and waiting for the IF bit to be set.
- The SS/CS line should be pulled low before you send the first 8 bits and pulled back high after you send the second 8 bits.
- Do not set SS/CS high in between sending the bytes.

Lab Practice #13: LED dimmer using a digital pot

Write a simple program to control a digital pot using the MCP4131.

- Configure the SPI in Host mode and connect the SPI to the MCP4131

- Using the potentiometer (E0), read ADC value and convert to the digital pot value
- Set output of the digital pot to the LED with a 330 ohms resistor.
- You can demonstrate the LED dimming using a digital pot, which is controlled by a potentiometer.